BS - KVL

Schwierigkeit von Prozessen

→ Prozesserzeugung, Prozessumschaltung sind teuer

- **Thread**

Ein Thread ist ein sequentieller Ausführungsstrang eines Prozesses, der sich mit den anderen Threads des zugehörigen Prozesses das Code- & das Datensegment teilt.

↳ Vorteile:

Nebenläufigkeit, Einfache Programmierung: weiterhin sequentielles Modell, gute Performance, billige Erzeugung und Kontext-wechsel, einfache Kommunikation mit Hilfe von Variablen, da gemeinsamer Adressraum.

↳ Thread ⇔ einzelner Programmzähler

⇝ Threads vs. Prozesse:

○ Prozesse bestehen aus: Code, Daten, Stapelspeicher, Dateitabelle, Adressierungsvektor-Tabelle, HW-Zustand

↳ nur Code kann gemeinsam benutzt werden

↳ Kommunikation über Pipes, Shared Memory, oder ähnlich aufwendige Mechanismen

↳ Threads können Informationen einfach, ohne BS, austauschen!

○ Threads nutzen gemeinsam: Daten, Dateitabelle, Adressierungsvektor-Tabelle

↳ Thread-Wechsel & -Erzeugung viel effizienter

BS - MSVB

→ pthread_create() → startet einen neuen Thread im aufrufenden Prozess
↳ start_routine wird aufgerufen

→ Thread Zustände:
- Alle Threads eines Prozesses: Speicherzustand, Dateisystemzustand
- Eines Einzelnen Threads: Programmzähler, Register, Thread-lokaler Stack, Zeiger auf Thread-lokalen Speicherbereich

Zustände:
- Erzeugung
  Wird neuer Prozess erzeugt, so wird Haupt-Thread erzeugt und in Bereit-Queue eingefügt, Haupt-Thread kann weitere Threads erzeugen
- Blockierung
  Wartet ein Thread auf ein Ereignis, wird er blockiert
- Aufhebung der Blockierung
  Wenn Ereignis auftritt, auf Grund dessen ein Thread blockiert war, dann kommt Thread in Bereit-Queue
- Beendigung
  Die belegten Ressourcen werden wieder freigegeben

BS - KAPITEL

- Simultaneous Multithreading (SMT)

Simultaneous Multithreading ist hardware-unterstütztes Multithreading bei dem ein physischer Prozessor in zwei logische Prozessoren aufgeteilt wird. Ein komplexes Steuerwerk & zwei vollständige Registersätze erlauben die Ausführung von zwei parallelen Befehlsströmen.

→ Threads: jeder Thread hat eigenen Stack ✓

BS-Kurz-Ex2

2-1

1) Was ist ein Signal?

Ein Signal ist eine Systemnachricht an einen laufenden Prozess.

2) Interrupt, Trap & Signal?

Ein Interrupt ist ein Signal, welches die normale Programmausführung unterbricht, um eine andere Verarbeitung durchzuführen

Trap ist ein Interrupt ausgelöst von einer Instruktion

3) Was ist ein Systemaufruf?

Ein Systemaufruf ist eine Methode, um alle vom BS bereitgestellten Funktionalitäten auszuführen. Dabei wird die Kontrolle vom Programm an den Kernel übergeben.

4) Was ist ein Thread?

Ein Thread ist ein sequentieller Ausführungsstrang eines Prozesses, der sich mit den anderen Threads des zugehörigen Prozesses das Code- & das Datensegment teilt.

BS-RVB – Ex2

(2-1)

5) Wodurch wird der Zustand eines Threads beschrieben?

Einzelner Thread: → Programmzähler
→ Register
→ Thread-lokaler Stack
→ Zeiger auf Thread-lokalen ~~Stack~~ Speicherbereich

6) Was passiert bei einer Umschaltung zwischen Threads im Gegensatz zu einer Umschaltung zwischen Prozessen?

Prozesswechsel: → TCB austauschen
→ Speicherverwaltungstabelle austauschen
→ Speicherbereiche ersetzen
→ Entleeren & Neuladen des Caches

Threadwechsel: → TCB austauschen

Threadwechsel viel effizienter

BS-RVB-Ex2

(2-2) Signale

Jeder Prozess besitzt Unterbrechungsvektor-Tabelle, in der spezifiziert ist wie auf ein auftretendes Signal reagiert werden soll. ↝ Verhalten kann durch sig.action verändert werden

a) Was passiert wenn das Signal SIGUSR1 auftritt & wie kann es erzeugt werden.

↳ auftreten ↝ output: "received signal: SIGUSR1"

↳ in Linux Shell mit: kill -USR1 pid
↳ pid of running process

oder im Program kill(pid, SIGUSR1)

b) Hat ein Signal auswirkung auf sleep()?

↳ SIGALARM

c) Was passiert, wenn ein weiteres Signal während der Abarbeitung der Signalbehandlungsroutine auftritt?

→ sigemptyset(&neue_action.sa_mask)
- ∀ Signale \ {SIGUSR1} sind zugelassen innerhalb der Signalbehandlungsroutine.
- alle anderen Signale führen zu einer Unterbrechung der aktuellen Signalbehandlungsroutine.

If: sigfillset(&neue_action.sa_mask)
↳ ∀ Signal während Signalbehandlungsroutine blockiert
↳ erst nach Behandlung ausgeführt

DS - HW - Ex 2

2-2 Signale

d) Wie kann sichergestellt werden, dass die Ausgabe "active" alle 5 Sekunden erfolgt.

```
...
while (true){
    [...]
    int seconds = 5;
    while ((seconds = sleep(seconds)) != 0);
}
```

BS-NVB-Ex2

(2-5) Thread

- Gibt es einen Unterschied zwischen dem Thread, der main() ausführt & dem neuen erzeugten Thread?
  ↳ Unterschiedlicher Stack & PC

- Gibt es zwischen Threads eine ähnliche verwandtschaftliche Beziehung wie bei Prozessen?
  ↳ Nein

- Was passiert wenn pthread_join() entfernt wird?
  Wenn der Prozess beendet wird, dann wird auch der andere Thread sofort beendet.

- Wann wird ein Prozess bzw Thread beendet?

Prozess ende: main-Funktion wird verlassen
exit() aufgerufen
letzter Thread beendet sich

Thread ende: Thread-Funktion verlassen
pthread_exit()
Prozess wird beendet